

МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ
ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»



Направление подготовки
44.03.04 Профессиональное обучение (по отраслям)
направленность (профиль)
«Информатика и искусственный интеллект в образовании»

Курсовая работа
по дисциплине «Прикладные информационные технологии»
«Разработка и контейнеризация веб-приложения для управления задачами с использованием Docker и автоматизация выпуска сертификатов Let's Encrypt»

Обучающегося 3 курса
очной формы обучения
Смирнов Михаил Борисович

Руководитель курсовой работы:
старший преподаватель кафедры ИТЭО
Аксютин Павел Александрович

Санкт-Петербург

2026

Оглавление

1. Введение	3
1.1 Актуальность темы исследования:	3
1.2 Цель исследования:	3
1.3 Задачи исследования:.....	3
1.4 Объект и предмет исследования	3
1.5 Методы исследования:.....	3
1.6 Практическая значимость.....	4
2. Теоретическая часть.....	5
2.1. Контейнеризация и платформа Docker	5
2.2. Docker Compose	5
2.3. Веб-сервер Nginx.....	5
2.4. Протокол HTTPS и Let's Encrypt.....	6
2.5. Сравнение и обоснование выбора технологий.....	6
2.5.1. Выбор фреймворка для серверной части (Backend).....	6
2.5.2. Выбор системы управления базами данных (СУБД).....	7
3. Программная реализация.....	8
3.1. Архитектура приложения	8
3.2. Серверная часть (Backend)	8
3.3. Клиентская часть (Frontend).....	8
3.4. Контейнеризация.....	8
3.5. Настройка Nginx и HTTPS	9
4. Анализ результатов	10
4.1. Тестирование приложения	10
4.2. Качество интерфейса и безопасность.....	10
5. Заключение	12
5.1 Основные результаты работы:	12
5.2 Практическая значимость:.....	12
6. Список использованных источников	13
7. Приложение	14
Приложение 7.1	14
Приложение 7.2	14

1. Введение

1.1 Актуальность темы исследования:

Сейчас при создании веб-приложений важно не только написать рабочий код, но и правильно его развернуть. Контейнеризация с помощью Docker решает проблему различий между средой разработки и сервером - приложение упаковывается вместе со всеми зависимостями и работает одинаково везде.

Кроме того, любое веб-приложение должно работать по защищённому протоколу HTTPS. Настройка SSL-сертификатов вручную занимает время, но этот процесс можно автоматизировать с помощью Let's Encrypt и Certbot.

Актуальность работы заключается в создании веб-приложения с использованием Docker-контейнеров и автоматической настройкой HTTPS.

1.2 Цель исследования:

Цель работы - разработать веб-приложение для управления задачами, упаковать его в Docker-контейнеры и настроить автоматический выпуск SSL-сертификатов Let's Encrypt.

1.3 Задачи исследования:

1. Разработать API на Python с использованием фреймворка Flask.
2. Создать интерфейс.
3. Написать Dockerfile и docker-compose.yml для запуска приложения.
4. Настроить Nginx как веб-сервер.
5. Настроить автоматическое получение SSL-сертификатов через Certbot.
6. Протестировать работу приложения.

1.4 Объект и предмет исследования

1. Объект исследования - процесс разработки и развёртывания веб-приложений.
2. Предмет исследования — технологии контейнеризации (Docker, Docker Compose) и методы настройки HTTPS (Let's Encrypt, Certbot)

1.5 Методы исследования:

1. Разработка REST API для связи клиентской и серверной частей приложения.
2. Контейнеризация приложения и настройка совместного запуска сервисов через Docker Compose.
3. Функциональное тестирование API и пользовательского интерфейса

1.6 Практическая значимость

Разработано рабочее веб-приложение, доступное по протоколу HTTPS. Конфигурация Docker Compose может использоваться как шаблон для развёртывания других проектов.

Обзор использованных технологий:

1. Python и Flask - для создания API.
2. HTML, CSS - для интерфейса.
3. SQLite - база данных.
4. Docker и Docker Compose — контейнеризация.
5. Nginx - веб-сервер.
6. Let's Encrypt и Certbot - SSL-сертификаты.

Выбор технологий обусловлен их простым и широким применением.

2. Теоретическая часть

2.1. Контейнеризация и платформа Docker

Контейнеризация - это метод изоляции приложений, при котором код и все необходимые ему зависимости (библиотеки, настройки, системные утилиты) упаковываются в единый образ. В отличие от традиционных виртуальных машин, контейнеры не эмулируют аппаратное обеспечение и используют ядро хостовой операционной системы. Это делает их значительно более легковесными, быстрыми в запуске и экономичными по потреблению ресурсов.

Docker - это самая распространенная платформа для работы с контейнерами. Главная проблема, которую решает Docker, заключается в различии сред разработки и продакшена. Приложение, упакованное в Docker-образ, будет работать абсолютно одинаково на ноутбуке разработчика, на тестовом стенде и на боевом сервере, что исключает ошибки, связанные с настройкой окружения.

2.2. Docker Compose

Современные веб - приложения редко состоят из одного компонента. Обычно это связка из бэкенда, фронтенда, базы данных и других сервисов. Запускать и настраивать каждый контейнер вручную через командную строку неудобно и подвержено ошибкам.

Docker Compose - это инструмент для описания и управления многоконтейнерными приложениями. Вся инфраструктура описывается в одном декларативном файле `docker-compose.yml` в формате YAML. В этом файле указываются используемые образы, пробираемые порты, общие сети и тома для хранения данных. Это позволяет развернуть всё приложение целиком с помощью всего одной команды, а также легко переносить его между серверами.

2.3. Веб-сервер Nginx

Nginx - это HTTP-сервер. В этом приложении он делает две вещи:

1. Раздаёт статические файлы. Nginx напрямую отдаёт браузеру HTML, CSS и JavaScript. Это быстрее, чем если бы файлы отдавал Python-бэкенд.
2. Все запросы к API бэкенда сначала приходят в Nginx, который перенаправляет их на внутренний порт бэкенда. Сам бэкенд при этом остаётся недоступным напрямую из интернета.

2.4. Протокол HTTPS и Let's Encrypt

HTTPS шифрует данные между браузером и сервером. Для работы HTTPS нужен SSL-сертификат.

С помощью утилиты Certbot сертификат получается автоматически после проверки владения доменом. Поскольку сертификаты выдаются на 90 дней, Certbot настраивается на автоматическое продление, что исключает ручное вмешательство

2.5. Сравнение и обоснование выбора технологий

При проектировании архитектуры веб-приложения и инфраструктуры его развертывания для каждого ключевого компонента системы был проведен сравнительный анализ существующих технологических решений на основе академических и инженерных критериев

2.5.1. Выбор фреймворка для серверной части (Backend)

В качестве альтернативы легковесному фреймворку Flask рассматривался полнофункциональный веб-фреймворк Django. Сравнительный анализ систем приведен в таблице 2.5.1

Таблица 2.5.1 – Сравнительный анализ веб-фреймворков Flask и Django

Критерий сравнения	Flask	Django
Архитектура	Микрофреймворк, модульная структура	Монолит («всё включено»), паттерн MTV
Избыточность	Минимальная, подключаются только нужные пакеты	Высокая для лаконичных REST API (встроенная админ-панель, ORM, авторизация)
Потребление ресурсов	Низкое, высокая скорость инициализации в контейнере	Более высокое из-за тяжеловесности ядра
Гибкость настройки	Высокая, разработчик сам формирует структуру проекта	Низкая, жестко заданная структура «из коробки»

Обоснование выбора: Использование Django для реализации лаконичного REST API с четырьмя базовыми CRUD-методами было признано нецелесообразным. Избыточный функционал фреймворка усложнил бы структуру Docker-контейнера и привел к нерациональному использованию ресурсов сервера. Выбор Flask обусловлен его легковесностью, высокой скоростью работы и возможностью гибко настроить только необходимые для веб-службы эндпоинты.

2.5.2. Выбор системы управления базами данных (СУБД)

В качестве альтернативы реляционной файловой базе данных SQLite рассматривались клиент-серверные промышленные СУБД PostgreSQL и MySQL. Сравнительный анализ систем приведен в таблице 2.5.2

Таблица 2.5.2 – Сравнительный анализ систем управления базами данных

Критерий сравнения	SQLite	PostgreSQL / MySQL
Тип архитектуры	Встраиваемая, данные хранятся в одном файле	Клиент-серверная, требует постоянного сетевого демона
Инфраструктура Docker	Не требует отдельного контейнера (монтируется через Volume к Backend)	Требует развертывания отдельного изолированного контейнера
Администрирование	Нулевое, не требует настройки пользователей и прав	Сложное: необходима конфигурация доступов, паролей и портов
Потребление ресурсов	Минимальное, использует общую память с процессом бэкенда	Высокое, требует выделения RAM и CPU под отдельный сервис

Обоснование выбора: Поскольку главной задачей исследования являлось проектирование и оптимизация инфраструктуры развертывания (связки веб-сервера Nginx и утилиты Certbot), усложнение архитектуры данных за счет развертывания отдельного тяжелого контейнера для клиент-серверной СУБД было избыточным. SQLite обеспечивает необходимую производительность для учебного прототипа, не требует оперативной памяти под отдельный сервис и надежно сохраняет состояние базы данных при перезапуске инфраструктуры за счет использования стандартных механизмов Docker Volumes.

3. Программная реализация

3.1. Архитектура приложения

Приложение построено по клиент-серверной архитектуре. Оно состоит из трех независимых компонентов: серверной части, клиентской части и базы данных. Компоненты работают в изолированных Docker-контейнерах.

3.2. Серверная часть (Backend)

Серверная часть написана на Python с использованием фреймворка Flask. Она отвечает за обработку входящих запросов и работу с данными. Для хранения задач используется SQLite - легковесная база данных, которая хранит информацию в обычном файле и не требует запуска отдельного сервиса.

API поддерживает четыре основных метода:

1. GET /api/tasks - получить список всех задач.
2. POST /api/tasks - создать новую задачу (принимает название, приоритет и срок).
3. PUT /api/tasks/<id> - изменить статус задачи (выполнена/не выполнена).
4. DELETE /api/tasks/<id> - удалить задачу по её идентификатору.

3.3. Клиентская часть (Frontend)

Интерфейс реализован в виде веб-страницы на HTML и CSS. Внешние фреймворки и тяжелые библиотеки не применялись.

Взаимодействие с сервером происходит напрямую через REST API. При добавлении, удалении или изменении задач интерфейс обновляется мгновенно, без полной перезагрузки страницы. Также на стороне клиента реализована фильтрация списка задач (все, активные, выполненные) и подсчет общей статистики.

3.4. Контейнеризация

Для упаковки серверной части используется Dockerfile. Он устанавливает зависимости и запускает его на внутреннем порту.

Управление всеми сервисами настроено в файле docker-compose.yml. В нем описаны три контейнера:

1. backend — запускает API.
2. frontend — запускает веб-сервер Nginx и открывает внешние порты 80 и 443.
3. certbot — фоновый контейнер для автоматического продления SSL-сертификатов.

Для сохранения данных базы данных при перезапуске контейнеров используется Docker volume. Это гарантирует, что задачи не будут потеряны, если контейнер с бэкендом будет пересоздан.

3.5. Настройка Nginx и HTTPS

Для реализации HTTPS настроен автоматический выпуск SSL-сертификатов от Let's Encrypt через Certbot.

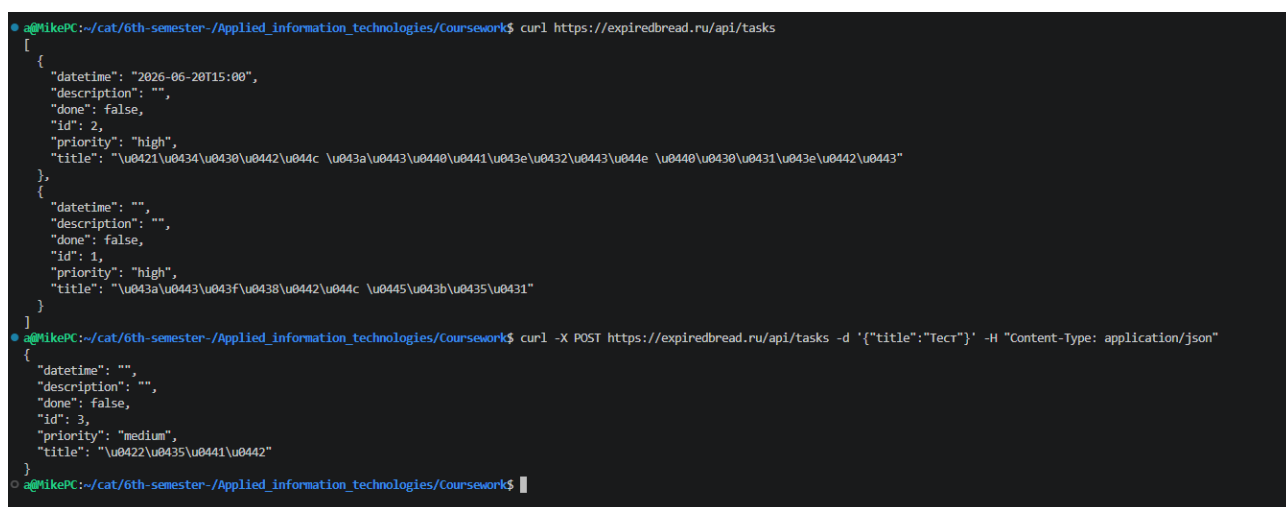
1. Проверка домена: Certbot запрашивает сертификат для expiredbread.ru и помещает проверочный токен в общую директорию. Nginx принимает запрос от Let's Encrypt на 80-м порту и отдаёт этот файл, подтверждая владение доменом, после чего генерируются SSL-ключи. Связь контейнеров настроена через Docker Volumes.
2. Двухэтапный запуск: чтобы обойти проблему зависимостей (Nginx не запускается по HTTPS без ключей, а Certbot не выдаёт их без работающего Nginx), сначала веб-сервер поднимается только на 80-м порту для прохождения проверки. После получения первых сертификатов конфигурация переключается на порт 443, и Nginx перезапускается без остановки системы.
3. Автопродление: Контейнер Certbot в фоне каждые 12 часов выполняет команду `certbot renew`. Если до истечения 90-дневного срока действия сертификата остаётся менее 30 дней, он обновляется, а Nginx автоматически перезагружает конфигурацию.

4. Анализ результатов

4.1. Тестирование приложения

Приложение было протестировано как на уровне серверного API, так и через пользовательский интерфейс. Тестирование проводилось на развернутом приложении по адресу <https://expiredbread.ru>

С помощью утилиты curl проверялись все методы REST API: получение списка задач, создание новой задачи, изменение её статуса и удаление. Все запросы возвращают корректные ответы в формате JSON, данные успешно записываются в базу данных SQLite.



```
a@MikePC:~/cat/6th-semester-/Applied_information_technologies/Coursework$ curl https://expiredbread.ru/api/tasks
[
  {
    "datetime": "2026-06-20T15:00",
    "description": "",
    "done": false,
    "id": 2,
    "priority": "high",
    "title": "\u0421\u0434\u0430\u0442\u044c \u0430\u0431\u0438\u0442\u0440\u0430\u0442\u0443\u0440\u044b \u0434\u043b\u044f \u0432\u0435\u0431\u0438\u043d\u0430\u0440\u0430"
  },
  {
    "datetime": "",
    "description": "",
    "done": false,
    "id": 1,
    "priority": "high",
    "title": "\u0430\u0434\u0430\u043f\u0442\u0430\u0442\u044c \u0434\u043e\u0434\u0435\u043b\u0430 \u0434\u043b\u044f \u0432\u0435\u0431\u0438\u043d\u0430\u0440\u0430"
  }
]
a@MikePC:~/cat/6th-semester-/Applied_information_technologies/Coursework$ curl -X POST https://expiredbread.ru/api/tasks -d '{"title":"Тест"}' -H "Content-Type: application/json"
{
  "datetime": "",
  "description": "",
  "done": false,
  "id": 3,
  "priority": "medium",
  "title": "\u0422\u0435\u0441\u0442"
}
```

Рисунок 4.1 – Процесс тестирования REST API с помощью утилиты curl

4.2. Качество интерфейса и безопасность

Интерфейс приложения минималистичен и загружается мгновенно, так как не использует тяжелые внешние библиотеки и системы сборки. Страница корректно отображается и адаптируется под экраны как настольных компьютеров, так и мобильных устройств.

С точки зрения безопасности, весь трафик между пользователем и сервером зашифрован по протоколу HTTPS. Серверная часть (бэкенд) изолирована внутри внутренней сети Docker и недоступна для прямых запросов из интернета - весь внешний трафик проходит только через Nginx.

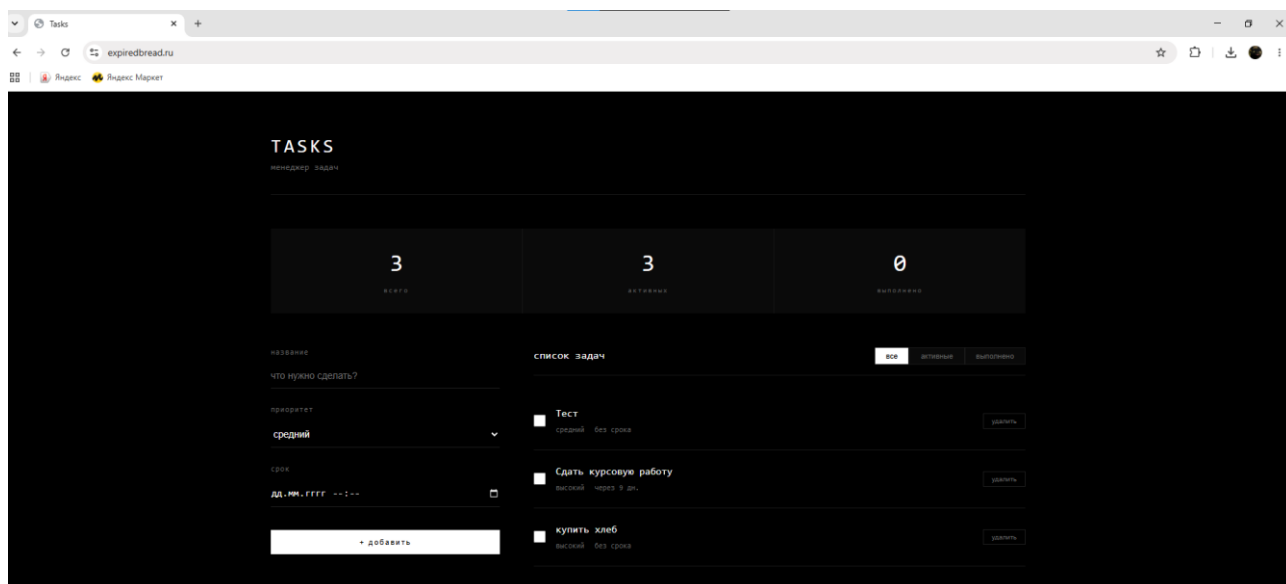


Рисунок 4.2 – Пользовательский интерфейс веб-приложения

5. Заключение

В ходе выполнения курсовой работы было разработано веб-приложение для управления задачами с использованием Docker-контейнеров и автоматической настройкой HTTPS. Приложение развернуто на удаленном сервере и доступно по адресу <https://expiredbread.ru>



Рисунок 5.1 – QR-код для перехода к развернутому веб-приложению

5.1 Основные результаты работы:

1. Разработано веб-приложение, включающее серверную часть на Python/Flask и клиентский интерфейс на HTML/CSS.
2. Приложение упаковано в Docker-контейнеры и настроено для совместного запуска через Docker Compose.
3. Настроен веб-сервер Nginx.
4. Реализована автоматическая выдача и обновление SSL-сертификатов через Let's Encrypt и Certbot.

5.2 Практическая значимость:

Разработанное приложение доступно по протоколу HTTPS и готово к использованию. Конфигурация Docker Compose может служить шаблоном для быстрого развертывания других веб-проектов с автоматической настройкой безопасности.

6. Список использованных источников

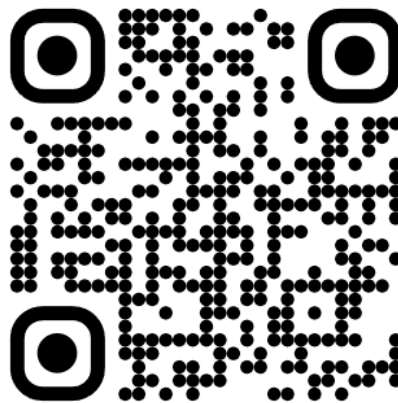
1. Docker documentation : официальный сайт. – URL: <https://docs.docker.com/> (дата обращения: 14.05.2026). – Текст : электронный.
2. Docker Compose documentation : официальный сайт. – URL: <https://docs.docker.com/compose/> (дата обращения: 18.05.2026). – Текст : электронный.
3. Flask documentation (Rules) : сайт. – URL: <https://flask.palletsprojects.com/> (дата обращения: 20.05.2026). – Текст : электронный.
4. Nginx documentation : сайт. – URL: <https://nginx.org/en/docs/> (дата обращения: 25.05.2026). – Текст : электронный.
5. Let's Encrypt documentation : сайт. – URL: <https://letsencrypt.org/docs/> (дата обращения: 02.06.2026). – Текст : электронный.
6. Certbot Instructions : сайт. – URL: <https://certbot.eff.org/instructions> (дата обращения: 04.06.2026). – Текст : электронный.
7. Python documentation : официальный сайт. – URL: <https://docs.python.org/3/> (дата обращения: 15.05.2026). – Текст : электронный.
8. SQLite Documentation : сайт. – URL: <https://www.sqlite.org/docs.html> (дата обращения: 22.05.2026). – Текст : электронный.
9. HTTPS — MDN Web Docs : руководство / Mozilla Developer Network. – URL: <https://developer.mozilla.org/en-US/docs/Web/HTTP/HTTPS> (дата обращения: 28.05.2026). – Текст : электронный.
10. Nginx Beginner's Guide : руководство. – URL: https://nginx.org/en/docs/beginners_guide.html (дата обращения: 08.06.2026). – Текст : электронный.

7. Приложение

Приложение 7.1

Полный исходный код разработанного веб-приложения, включая конфигурационные файлы Docker, настройки Nginx и серверную часть на Python, доступен в репозитории на платформе GitHub.

Адрес репозитория: <https://github.com/KOTorCAT/Coursework>



7.1.1 – QR-код, ведущий на репозиторий с кодом

Приложение 7.2

Внешний вид разработанного веб-приложения

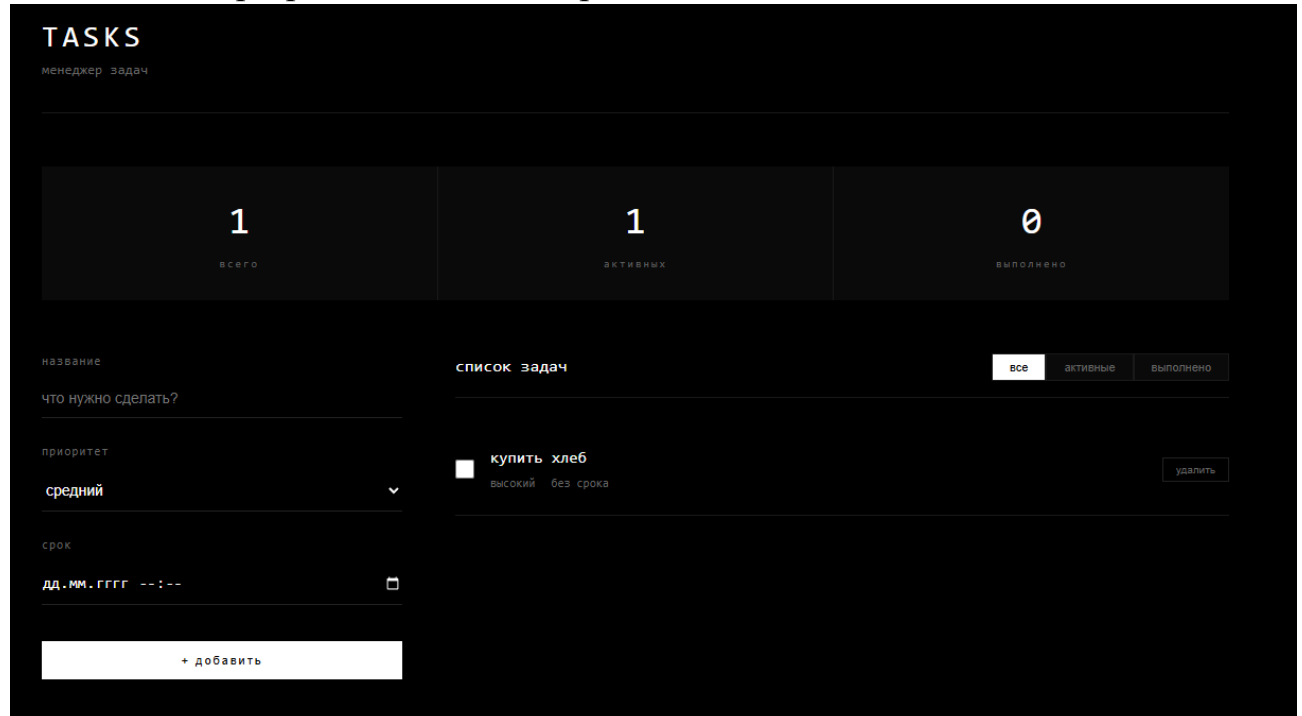


Рисунок 7.2.1 – Главный экран приложения с задачами